CloudGuardian Architecture & Working Conditions

CloudGuardian is a Docker resource management platform that exposes a web interface and REST API to manage containers, images, volumes, networks, services, stacks, and Docker Swarm resources. It uses a Node.js backend, a React frontend, PostgreSQL, Prisma ORM, and Docker daemon access through Dockerode.

1. High-level architecture

• Frontend: React + Vite + Material UI, served on port 8080

• Backend: Express.js REST API, JWT auth, Swagger UI, rate limiting, logging, served on port 5000

• Database: PostgreSQL 16, accessed through Prisma ORM

• Docker integration: Dockerode talks to the local Docker daemon / connected Docker hosts for container resource operations

• Deployment: Docker Compose runs postgres, backend, and frontend as separate containers with health checks and restart policies

2. Runtime flow

1. User opens the frontend in the browser.

2. Frontend requests API endpoints through Axios for auth, projects, containers, services, stacks, etc.

3. Express backend validates the request, applies CORS, Helmet, rate limiting, and JWT checks where required.

4. Prisma reads or writes application data to PostgreSQL.

5. For Docker operations, the backend uses Dockerode to send requests to Docker hosts and retrieve container, image, network, volume, and swarm data.

6. The API returns JSON to the frontend, which renders dashboards and management screens.

3. Main project structure

frontend	React app (Vite + React Router + MUI)
backend	Express API and business logic
backend/src/routes	Route registration for auth, users, projects, docker hosts, containers, etc.
backend/src/controllers	Request handlers
backend/src/middleware	Auth, validation, error handling, logging, rate limiting
backend/src/config	Prisma and Swagger configuration

backend/prisma	Schema and migrations for PostgreSQL
docker-compose.yml	Starts Postgres, backend, and frontend containers
scripts/	Backup and health check scripts for Windows

4. Working conditions / setup requirements

• Docker and Docker Compose installed on the machine or server

• Node.js 18+ for local development

• PostgreSQL database available to the backend or via Docker container

• .env values such as POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB, JWT_SECRET, JWT_EXPIRES_IN, and CORS_ORIGIN

• The backend checks database connectivity on startup and exits if PostgreSQL is unavailable

• Docker Compose health checks ensure backend starts only after PostgreSQL is healthy and frontend starts only after backend is healthy

• The app exposes: frontend on http://localhost:8080, backend on http://localhost:5000, Swagger at http://localhost:5000/api/docs

5. Local startup commands

```
cd backend

npm install

npx prisma generate

npx prisma migrate dev

npm run dev

docker compose up --build -d
```

6. Operational notes

• Backend uses JWT auth, so protected routes require Authorization: Bearer .

• The backend health endpoint is /health and verifies database connectivity.

• For HTTPS in production, use the optional Caddy profile and publish ports 80 and 443.

• For production hardening, keep secrets in a manager, restrict CORS origin, and perform regular database backups.

Generated for project review: CloudGuardian